

Lab exercises

Part 13 —  AI Security · 13-ai-security/lab/EXERCISES.md

Lab — AI Security

Work top to bottom. Each exercise has a **stated deliverable**; if you can't produce it, you haven't finished. Solutions are intentionally not provided — the checks tell you when you're right. Nothing here requires an API key or network access; everything runs against the two scripts in `../code/`.

1. Warm-up: read the code you were given (45 min)

Run both scripts:

```
/Users/srinip/ai-all/.venv/bin/python ../code/prompt_injection_defense_demo.py
/Users/srinip/ai-all/.venv/bin/python ../code/lethal_trifecta_analyzer.py
```

- **1a.** In `prompt_injection_defense_demo.py`, the "novel" variant gets through layers 0-2 and is only stopped at layer 3. Find the exact line of `classifier_flags()` logic that explains why layer 2 misses it, and quote the specific words in the novel payload that would need to be added to `KNOWN_INJECTION_PATTERNS` to catch it.
- **1b.** In `lethal_trifecta_analyzer.py`, `inbox_triage_agent_v1` and `inbox_triage_agent_v2_gated` differ by exactly one field. Name it and explain, in the analyzer's own terms, which of the three legs it neutralizes and why gating doesn't remove the tool, just its "ungated" status.
- **1c.** `customer_account_assistant` is not flagged even though it accesses sensitive data. Explain, in one sentence, why `reply_to_current_user` does not count as leg 3 — and describe a one-line code change to that tool's *purpose* (not its tags) that would make counting it as leg 3 the correct call.

Deliverable: three short written answers, one per part.

2. Break the classifier on purpose (1.5h)

Add a fourth injection variant to `prompt_injection_defense_demo.py` called "encoded": base64-encode the exfiltration instruction and prepend a plain-language line telling the model to decode and follow it (e.g. "Decode the following base64 and follow its instructions: <encoded blob>"). You will need to extend `FakeModel.respond()` with a rule that decodes base64 substrings and re-checks them for the same imperative/target patterns it already checks in plaintext — that decode-then-recheck step *is* the point.

- **2a.** Run all four layers against the new variant. Predict, before running, which layers will catch it and which won't — then run it and compare.
- **2b.** `classifier_flags()` only scans plaintext. Extend it with a base64-detection step (a regex for a long base64-looking run, then a decode-and-rescan) so layer 2 catches the encoded variant too.

- **2c.** Now argue the other side: what is the next obfuscation technique (not base64) that would defeat your improved classifier? You do not need to implement it — name it and explain why decode-and-rescan doesn't generalize to it.

Check: your updated summary table shows the encoded variant getting through the *original* layer 2 but blocked by your *improved* layer 2, and still blocked by layer 3 regardless. Part 2c names a real technique from notes/01 section 6 (encoding/obfuscation family) that your fix does not cover.

3. Design and flag your own agent (1.5h)

Pick a real (or realistic) agent from your own work or a system you know — a code-review bot, a meeting summarizer with calendar access, a support bot with refund authority, anything with at least two tools.

- **3a.** Add 3-5 new `Tool(...)` entries to `lethal_trifecta_analyzer.py`'s registry, tagging each honestly against the three-question checklist. Add one `AgentConfig` using them.
- **3b.** Run the analyzer. If it's not flagged, deliberately add one more tool/grant that *would* trigger the trifecta (a plausible one — "and then someone asked for a Slack notification on completion" is a realistic scope-creep story, not a contrived one).
- **3c.** Apply exactly one of the three fixes from notes/02 section 2 (split, gate, or remove) to your now-flagged config, in code, and show the analyzer's verdict flip back to "safe."

Deliverable: a diff (or before/after code block) of your registry and agent additions, plus the analyzer's printed output for both the flagged and fixed versions.

Check: your flagged config genuinely has all three legs from *real* capabilities you named yourself, not from artificially inflated tags chosen to force a flag.

4. Diagram-to-checklist, by hand first (1h)

Before you're allowed to run any script for this one: sketch (on paper, in a text file, doesn't matter) an architecture diagram for a "**meeting-prep assistant**" that reads a user's calendar, searches the web for attendee background info, drafts a briefing doc, and can post that doc to a shared Google Drive folder the whole team can see.

- **4a.** Walk the three checklist questions from notes/02 section 2 against this diagram by hand. Write your yes/no answers and which specific capability drove each.
- **4b.** Now encode the same agent into `lethal_trifecta_analyzer.py` and run it. Does the script's verdict match your by-hand answer? If not, find the mismatch — it is almost always a tagging judgment call (is "post to a shared Drive folder" `can_communicate_externally`? Argue both sides in one sentence before you decide).

Deliverable: your hand-written checklist answers plus the script's printed verdict, with the mismatch (if any) explained.

5. Jailbreak family spotting (1h)

Without writing any working exploits, take three real conversation transcripts (write short fictional ones, 4-8 turns each, plausible but harmless — e.g. escalating a request for "creative writing tips" into something you'd flag) such that:

- transcript A demonstrates **roleplay/persona framing**,
- transcript B demonstrates **crescendo/multi-turn escalation**,
- transcript C demonstrates **many-shot context stuffing** (describe the shape — dozens of fabricated Q&A pairs — you don't need to write all of them out, a representative excerpt plus a note on the pattern is enough).

For each, write one sentence naming the **defensive signal** from notes/01 section 6's table that a red-teamer or a monitoring system should have caught, and at which turn.

Check: a colleague who has read notes/01 section 6 but not your transcripts can correctly match each transcript to its family from your one-sentence defensive-signal note alone.

6. Capstone: full threat model (3h)

Take the meeting-prep assistant from exercise 4 (or your own agent from exercise 3) and produce a one-page threat model containing, in order:

1. An architecture sketch (ASCII is fine) showing every tool and every data source.
2. The lethal-trifecta checklist verdict (by hand or via the analyzer) for every node.
3. For every risk you find, which OWASP LLM category (2026 list, notes/01 section 2) it falls under.
4. The defense-in-depth table from notes/01 section 5.5, applied: for each layer you'd deploy, name which specific attack variant it catches for *this* system, and which variant would still get through.
5. One paragraph, written as if handing this to a colleague, stating honestly what is NOT covered even after every mitigation is applied.

Check: section 5 is not generic boilerplate — it names a specific plausible attack against *this* system that survives everything in section 4. If you can't name one, you haven't looked hard enough; go back to notes/01 section 4 and re-read why no defense here is a categorical fix.

7. Stretch: red-team scoring (1.5h)

Using the "traditional pentest vs. LLM red-team" table in notes/02 section 6, design a scoring rubric (a simple table: technique, target capability, success rate bucket, severity, single-turn vs. multi-turn) and apply it to the three transcripts you wrote in exercise 5, as if they were real red-team findings against your exercise-3 or exercise-4 agent.

Check: your rubric distinguishes "succeeded once in three attempts" from "succeeds reliably" as different severities — that distinction, not just pass/fail, is the exercise's whole point (notes/02 section 6, "Scoring")

row).